Measuring the Overhead of Trust Mechanisms in the X-Road Data Exchange Layer

Ruslan Nechshadimov
University of Tartu
Supervisor: Artjom Lind

Draft v0.1, September 17, 2026

Rude draft compiled with AI help (thoughts and idea is mine, AI just compile). I really hate how this is written, especially the second and third paragraphs of the introduction, but I think it is enough to understand the idea.

Abstract

A doctor opening a patient's prescription history, a notary checking who owns a property and a tax official pulling a company's filings all need the same thing: an answer that is not only correct but provable. The recipient has to know which organisation produced the record, that it was not altered on the way, and that the sender cannot later deny having sent it. In Estonia and in a growing number of national ecosystems these guarantees are provided by X-Road, the data exchange layer between the information systems of the participating organisations. They are produced by concrete mechanisms — a digital signature on every message, verification on the receiving side, a message log, time stamps issued by an external authority, a mutually authenticated channel and certificate status checking — and each of them consumes time and bytes on every exchange. This report proposes a measurement study that decomposes the end-to-end cost of an X-Road exchange into the contributions of these individual mechanisms. We build a testbed of one Central Server, a test certification authority providing certificate status and time-stamping services, four Security Servers and two information systems; we compare a ladder of deliberately varied trust configurations against a baseline that does not use X-Road at all; and we repeat the experiment in several environments, from a single laptop to physically separated machines connected by a real network. The expected result is a per-mechanism cost table in milliseconds and bytes, with raw data and a redeployable testbed, which can serve as the reference point that current proposals for post-quantum, decentralised public-key-infrastructure and threshold-signature variants of X-Road currently lack.

1 Introduction

When a hospital queries a registry over X-Road, a sequence of protective steps runs before any data is returned. The request is signed with the member organisation's signing key and verified on the other side; both Security Servers write the message into their message logs; the log records are time-stamped by an external Time-Stamping Authority (TSA); the channel between the two servers is a mutually authenticated Transport Layer Security (TLS) connection; and the certificates involved are checked against responses obtained through the Online Certificate Status Protocol (OCSP) [10, 4]. Together these mechanisms give integrity, authenticity and non-repudiation, which is what allows an exchange to be treated as evidence. They are also what the system pays for that property, on every single message.

What is that price? Measurements of X-Road do exist, but they were designed to answer other questions and therefore do not decompose it. The most recent figures come from a study of distributed key management, which reports round-trip times for a complete exchange on X-Road

7.7.0 — about 32 ms with the built-in software token, rising to 3081 ms for an 11-of-15 threshold group [3]. For that study an aggregate number is exactly the right instrument, since the question was whether threshold signatures are competitive with single-party tokens; the authors also note that all components ran in containers on one laptop and present their results as a lower bound rather than production performance. A second study compares calls routed through a Security Server with calls sent directly to a service adapter and reports differences of 56 to 319 ms at ten concurrent clients [6]. Its goal was capacity planning for a particular registry service, so database time dominates by construction; at twenty and thirty concurrent clients the path through the Security Server is reported as faster than the direct one, which indicates that at those load levels the variation of the environment was comparable to the difference between the two paths. Finally, the architectural decisions that determine much of the cost — caching OCSP responses rather than querying per signature, moving time-stamping to the receiver and batching it over a hash tree — were introduced together with an analytical estimate of 11.84 MB/s for batch signing, and that paper explicitly leaves benchmarking of the implementation to future work [1]. We have not found a published follow-up. In short, the decomposition we are after has not been anyone's research question so far.

Two developments make the question more pressing. The first is volume. The design reasoning above was calibrated against a load of roughly 240 million queries per year [1]; the operator's public factsheet reports 3.74 billion queries in the last year, about 1000 member organisations and 2083 connected information systems [9]. Whatever is paid per message is now multiplied by a substantially larger base. The second is the planned migration to post-quantum cryptography. A recent report by the Nordic Institute for Interoperability Solutions (NIIS) inventories 27 cryptographic assets in X-Road 7.6.2 and expects hybrid and post-quantum signatures to enlarge certificates, signatures and handshakes, to cause message fragmentation and additional round trips, and to require replacing Hardware Security Modules (HSMs) that cannot hold post-quantum keys; it contains no measurements of its own [4]. An earlier Cybernetica report on the commercial Unified eXchange Platform reaches a similar conclusion from the primitive side, listing, for example, a 4595-byte Dilithium5 signature against the 256 bytes of RSA-2048, and recommends testing hybrid modes in the Security Servers [5]. Both describe what will become more expensive; neither can express by how much, because there is no measured baseline for the present system. The same applies to decentralised Public Key Infrastructure (PKI) built on verifiable credentials, whose proof-of-concept adds a credential presentation to every request and whose performance was explicitly left unevaluated [2].

This project produces that baseline. The research questions are:

RQ1
: How much does each trust mechanism — TLS between Security Servers, message signing, signature verification, certificate status checking, time-stamping and message logging — contribute to end-to-end latency and to the number of bytes on the wire?

RQ2
: How do those contributions change with message size and with concurrency?

RQ3
: How much do the answers depend on the environment: all components on a single host, separate machines connected by a local network, or nodes separated by a wide-area link?

RQ3 is included because the published measurements were taken with all components on one machine, a limitation their authors state themselves. If the relative weight of cryptography turns out to be an artefact of the absence of a network, that is worth knowing before such numbers are used to reason about migration.

The intended contribution is useful rather than novel in method: a redeployable multi-node testbed, a per-mechanism cost table in milliseconds and bytes obtained in several environments,

the raw per-request data, and a clearly labelled extrapolation of what a post-quantum signature scheme would do to the measured signing and verification share.

2 Background: what is paid per message

X-Road is a centrally managed distributed data exchange layer. A Central Server distributes the global configuration — the member registry, the trusted Certification Authorities (CAs) and the trusted time-stamping authorities — and each member operates a Security Server that handles key management, signing, secure transport, logging and access control [2, 10].

For this study the mechanisms divide by their position relative to the request path:

- **Synchronous**, paid on every message: the mutually authenticated TLS channel between the Security Servers; signing with the member's signing key; verification of the incoming signature; and writing request and response into the message log, whose bodies are logged in full by default [11]. How much the TLS channel costs per message depends on whether connections are reused [12]. By default they are not: the client-side pool reuse (`pool-enable-connection-reuse`) and the provider-side support for pooled connections (`server-support-c`) are both off, so each message opens a new TCP connection and performs a TLS handshake [11]. In our traffic captures this is a TLS 1.2 handshake resumed from a session ticket, without a certificate exchange, which costs one round trip on top of the TCP one.
- **Asynchronous**, amortised or deferred: certificate status checking, which uses cached OCSP responses refreshed on a fixed interval (20 minutes by default) rather than one query per message; time-stamping, applied in batches over a hash chain of log records (once per minute by default, with a tolerated time-stamping outage of four hours, the parameter `acceptable-timestamp-failure` being 14400 seconds); and the periodic refresh of the global configuration [8, 11, 1].

The split is a documented design decision: caching status responses and deferring time stamps were introduced to keep a per-message PKI operation from becoming both a single point of failure and a latency source [1]. It carries a security price as well, since a cached, nonce-less OCSP response leaves a window during which a revoked certificate is still accepted [2]. The performance side of that trade-off is what we measure.

Two further properties shape the experiment. First, messages are signed individually while time-stamping is batched over hash chains [10, 1], so the per-message cost of time-stamping depends on load and has to be observed across concurrency levels rather than at a single point. Second, the intervals above are defaults, and a production deployment may use longer ones: for the Estonian instance of 2017 a master's thesis documents a global configuration validity of six hours and an OCSP freshness of eight hours [7]. Whether to cite those production values alongside the documented defaults is one of the questions in Section 6.

3 Methodology

3.1 Testbed

The testbed consists of one Central Server, a test CA providing certificate status and time-stamping services, four Security Servers (a management server, one service consumer and two service providers) and two information systems. Four Security Servers rather than the usual two give a genuinely distributed deployment in which the consumer's traffic is directed at two independent providers, so that queueing at a single node is not mistaken for protocol cost.

The provider-side information system is an echo service returning a response of configurable size with negligible processing time. This is deliberate: in the closest existing study database time dominated and the effect of interest was not separable from it [6]. The trade-off is that our

figures overstate the *relative* share of the trust mechanisms compared with a realistic service, so one calibration point uses an echo service that delays its response by 50 ms.

All components run as containers pinned by image digest, from a single composition used in every environment, with identical CPU and memory limits per container, so that environments differ in topology and network rather than in how much hardware each component receives.

3.2 Environments

The original intention was simpler: develop and debug the stack on a laptop, and collect the actual data on a server. Working through the plan showed that the choice of environment is not a detail of execution but part of RQ3, and that there are four variants worth considering, with different costs and different lead times. They are complementary rather than alternatives, and they can be dropped from the bottom of the list if the scope turns out to be too large.

A. Single host (laptop). All components on one machine (64 GB RAM available; per-container limits of 2 cores and 4 GB are applied so that results remain comparable with the other environments). There is no network between the Security Servers. This reproduces the conditions of the published measurements [3] and makes our numbers directly comparable with them. *Pitfalls:* the Windows/WSL2 stack adds overhead of its own, and the memory given to WSL2 has to be fixed in configuration, otherwise it varies between runs; the load generator competes with the containers for cores at higher concurrency and must be constrained and checked not to be the bottleneck.

B. Two or three laptops on a local network. Consumer and provider Security Servers on physically separate machines, the supporting components on a third machine or alongside the consumer. Requirements: machines of broadly comparable class, a wired switch rather than Wi-Fi, Docker on each, and about 50 GB of free disk on the machines hosting a Security Server. *Pitfalls:* hardware is heterogeneous, so part of any asymmetry between nodes is explained by the machines rather than by the network; measurements are hard for anyone else to reproduce; the network delay obtained here (a fraction of a millisecond) is of the same order as inside a data centre, so this variant adds a real network but not a different class of latency. Its main value is availability — it needs no approvals and can be used immediately to debug the stack while access to the cloud is being arranged.

C. Cloud virtual machines (University of Tartu High Performance Computing centre). The primary environment for the reported results, because the hardware is uniform, described, and free of foreground activity, which makes the runs reproducible by others.

- *C1, two or three virtual machines.* Consumer and providers on different machines, so the measured link exists. Indicative sizing: a core machine (Central Server, test CA, management Security Server) with 4 vCPU, 8 GB RAM and 40 GB disk; a consumer machine with 4 vCPU, 8 GB and 60 GB; a provider machine with 6 vCPU, 12 GB and 60 GB. Software: Ubuntu 24.04 LTS and Docker; TCP 5500, 5577, 80, 443 and 4001 reachable between the machines; the default Docker bridge network moved off 172.17.0.0/16, as the operator's documentation requires.

- *C2, a single virtual machine (fallback).* Everything on one instance of 8 vCPU, 32 GB RAM and 60 GB disk, no externally opened ports. There is no network between the Security Servers, so RQ3 is addressed only through emulated latency (`tc netem` [15], 1–50 ms), as in TLS benchmarking where emulation isolates the effect of latency and does not need Internet-scale infrastructure [14]. It is a weaker answer than a real wide-area link and is reported as such.

Pitfalls: lead time for the virtual machines and for opening the ports is outside our control, which is why the request is sent on day one and why variant B exists.

D. Hybrid: consumer on a laptop, the rest in the cloud (optional). The only variant with a wide-area link and therefore with a round-trip time an order of magnitude above the others. It requires no inbound access to the laptop, since the consumer always initiates the connection; it does require TCP 5500 and 5577 to be reachable on the cloud side, with a WireGuard tunnel as a fallback if that is not granted — in which case a control run with and without the tunnel is needed to separate the tunnel's own cost. *Pitfalls:* a home uplink is variable, and the variance may well be comparable to the effects we measure, as happened in [6]. This variant is therefore treated as an external-validity check rather than a source of primary results.

If the scope has to be reduced, D is dropped first and B second; A together with C is the minimal meaningful set.

3.3 Isolating the cost of a mechanism

Signing cannot be switched off in X-Road, so the decomposition is obtained by differencing a ladder of configurations:

1. **Direct HTTP** from the client to the echo service, without X-Road. Lower bound.
2. **Plain mutual TLS** between two simple proxies using the same certificates. Separates the cost of the channel from the cost of X-Road.
3. **X-Road, default configuration.** The reference point for normal operation.
4. **X-Road without time-stamping**, with `acceptable-timestamp-failure-period` set to zero and the time-stamping service disabled, a configuration in which the Security Server has been observed to keep processing requests independently of the TSA [7].
5. **X-Road with immediate time-stamping** (`timestamp-immediately`), which removes batching and exposes the full per-message cost.
6. **X-Road with a short OCSP freshness interval**, so that status checking is forced into the request path.
7. **X-Road with elliptic-curve keys** (secp256r1) instead of the default RSA-2048.

Differences (3)−(2) and (2)−(1) give the cost of X-Road above plain mutual TLS and the cost of the channel itself; (5)−(3), (6)−(3) and (3)−(4) give time-stamping and status checking explicitly. Configurations 4 to 6 are deliberately set away from their defaults in order to make a mechanism observable, and the figures obtained from them describe an upper bound on its cost; configuration 3 is what a deployment actually runs, and production deployments may use even longer intervals [7].

In parallel, X-Road's own operational monitoring records per-request timestamps and message sizes inside the Security Server, which we intend to use as an independent, non-differential decomposition. The exact fields and the query interface are to be confirmed against the protocol specification before the first run.

3.4 How deep to go into signature schemes

A deliberate scoping decision, and one where the supervisor's opinion is wanted. We do not intend to study the internals of signature schemes, prove anything about them, or implement one. A signature is treated as a component with a measurable cost profile: time to sign, time to verify, bytes added to the message, behaviour under concurrency.

Inside X-Road two schemes can be exercised today, since both are supported for signing and authentication keys: RSA-2048 and the Elliptic Curve Digital Signature Algorithm (ECDSA) over secp256r1 [4, 3]. Comparing them costs one configuration change and yields a second data point for the signing and verification share, which is what makes any extrapolation credible: a single measured scheme gives a number but not a slope.

Post-quantum schemes cannot be exercised inside X-Road at present; no implementation exists and both available reports treat the migration as future work [4, 5]. We therefore propose two layers. The system layer measures RSA-2048 and ECDSA in the real exchange. The primitive layer measures signing and verification of the Module-Lattice Digital Signature Algorithm (ML-DSA, formerly Dilithium) and Falcon — together with the corresponding RSA and ECDSA operations for calibration — on the same hardware, using an off-the-shelf library, so that the ratios we use are ours rather than borrowed from a benchmark on different hardware. The prediction for a post-quantum X-Road is then the measured per-message signing share scaled by the measured primitive ratio, plus the measured effect of message size on latency applied to the known growth in signature size (4595 bytes for Dilithium5 against 256 bytes for RSA-2048 [5]). Such a prediction is an extrapolation and will be labelled as one; its value is that each factor in it is measured rather than assumed. If a narrower scope is preferred, the primitive layer is the part to drop: the study remains sound with RSA and ECDSA alone and loses only the forward-looking section.

3.5 Factors, metrics and procedure

Payload sizes of 1 KB, 10 KB, 100 KB and 1 MB; concurrency of 1, 2, 4, 8, 16 and 32 clients; the seven configurations above; two key types; Representational State Transfer (REST) as the primary protocol. The grid is not fully crossed: sizes are crossed with configurations at a single client to obtain clean latency figures, and concurrency is crossed with configurations at 10 KB to obtain behaviour under load.

Metrics: median, 90th and 99th percentile latency (means are reported only for comparability with prior work); throughput at saturation; bytes on the wire per exchange; the number of status and time-stamping requests per thousand messages; message-log growth per message; and container-level CPU and memory use.

Each configuration is preceded by 50 warm-up requests and measured over 1000 requests, following [3]. Warm-up, independent repetitions and reporting the spread between them follow the practice for Java performance evaluation in [13], since the Security Server runs on the Java virtual machine. Measurements are repeated three times at different times of day, with the order of configurations randomised rather than run in blocks. At 1 MB payloads the count is reduced to 200 requests, which is sufficient at that size and keeps the message log from growing without need. Before the campaign a control run measures the same configuration twice in succession; if the difference between repetitions is comparable to the effect we expect, the testbed is not ready and the load or the node count is reduced. Raw per-request data, image digests and configuration files are published alongside the aggregates.

Storage deserves a note, because message bodies are logged in full by default on both Security Servers [11]. One exchange costs roughly twice the payload plus a few kilobytes of signature and metadata per server, so a thousand exchanges at 10 KB produce on the order of 60 MB in total and at 1 MB several gigabytes. The message log is therefore truncated between configurations and `keep-records-for` is set to one day, which keeps the peak requirement at a few gigabytes rather than letting it accumulate across the campaign.

3.6 Threats to validity

The test CA, status responder and TSA run next to the Security Servers, whereas in production they are external services with their own latency; our figures for those two mechanisms are

therefore optimistic. The echo service raises the relative share of the trust mechanisms, which the 50 ms calibration point mitigates but does not remove. The weakened configurations are upper bounds rather than descriptions of normal operation. Containerisation, and on the laptop the Windows/WSL2 stack, add overhead of their own, which is one of the reasons for running environments B and C at all.

4 Expected outcome

A table mapping each trust mechanism to milliseconds and bytes per message, obtained in several environments, supported by raw data and a testbed that can be redeployed from a single composition file. Beyond the seminar report, the result is a reference point for three lines of work that currently lack one: post-quantum migration [4, 5], decentralised PKI [2] and distributed key management [3], each of which proposes to make trust more robust at a cost that has not been compared with the cost of what X-Road does today.

5 Plan

1. Request virtual machines and ports from the university computing centre (day one; the rest of the work proceeds in parallel).
2. Composition file and testbed, identical across environments; automation of member and service registration.
3. Echo service, load scripts, metric collection, message-log cleanup, and the control run for stability.
4. Environment A, full grid.
5. Environment C (or B, whichever becomes available first), same grid, plus emulated latency.
6. Environment D, reduced grid, if it is kept in scope.
7. Analysis, figures, report.

Steps 2 and 3 are the schedule risk; the waiting time in step 1 is the reason environment B is on the list at all.

6 Questions for the supervisor

1. Signatures (Section 3.4): we can measure only what X-Road runs today, that is RSA-2048 and ECDSA, which is a configuration change and nothing more. The alternative is to also benchmark post-quantum primitives separately, on the same hardware, and use the ratio to predict what migration would do to the measured signing share. Is that second layer worth the extra work, or too speculative for a project of this size?
2. Proposal: run the main grid over REST and add a single SOAP (Simple Object Access Protocol) control point at 10 KB, so that the results remain comparable with older measurements made on SOAP services. Is that worth roughly one extra day, or should REST alone be used?
3. Environment B would need two or three machines of roughly comparable specification on a wired local network, with Docker, for about a week — I noticed a stack of laptops in your office. Would it be possible to borrow a few? To be clear about what this buys: it does not replace the cloud environment, where the hardware is uniform and the runs are reproducible by others, and it does not give the wide-area case either. It covers the middle level of RQ3,

that is whether the single-host results are an artefact of traffic never leaving one machine, and it lets the testbed be built and debugged while access to the cloud is still being arranged.

4. How quickly does the computing centre usually provide virtual machines and open non-default ports? The plan treats this as the longest lead time; if it is in fact short, environment B is needed only as a second physical environment rather than as insurance.

5. Which interval values should the report quote? The documentation gives defaults — a 20-minute OCSP cache and a 10-minute validity of the global configuration — while a 2017 master's thesis reports six and eight hours for the Estonian production instance [7]. The difference matters for interpretation: if real deployments keep these windows an order of magnitude wider, the mechanisms we deliberately weaken in order to measure them are even less visible in production than our upper bounds suggest. Mentioning the production values makes that point, but they come from a single secondary source that is nine years old.

6. Would it be appropriate to show the results to someone at NIIS or at the Estonian Information System Authority, and are there contacts for that? With a local test CA our figures are necessarily optimistic, and an informed opinion on whether the profile resembles production behaviour would strengthen the work considerably.

7. About the size of this. I looked at the seminar projects from previous years and they are all 4 to 6 pages, while this draft is already 8 pages without the actual work in it. Part of it may be formatting, but I have the feeling that I am simply doing more than the format expects. Doing less is not something I want, because then what is the point. This is not really a direct question, more thinking out loud — but if you have an opinion on it, I would like to hear it: is this scope fine as it is, or should it be cut, and if it should, how do I cut it without losing the meaning of the work?

References

[1] A. Ansper, A. Buldas, M. Freudenthal, J. Willemson. High-Performance Qualified Digital Signatures for X-Road. In *NordSec 2013*, LNCS 8208, pp. 123–138, Springer, 2013.

[2] M. Bakhtina, K. L. Leung, R. Matulevičius, A. Awad, P. Švenda. A Decentralised Public Key Infrastructure for X-Road. In *ARES 2023*, ACM. DOI: 10.1145/3600160.3605092.

[3] M. Bakhtina, J. Kvapil, P. Švenda, R. Matulevičius. Securing Organisational Digital Identity: A Full-Lifecycle Approach to Distributed Key Management. *International Journal of Information Security*, 25:131, 2026. DOI: 10.1007/s10207-026-01291-5.

[4] M. Bakhtina. Migration of X-Road to Post-Quantum Cryptography. NIIS Publication series, Nordic Institute for Interoperability Solutions, 2026. ISBN 978-9916-9853-4-2.

[5] N. Snetkov, J. Vakarjuk. Integrating Post-Quantum Cryptography to UXP. Cybernetica Research Report D-2-499, 2023.

[6] E. Taklai. X-tee teenuste jõudlustestimine Eesti äriregistri näitel (Performance testing of X-Road services on the example of the Estonian Business Register). Diploma thesis, Tallinn University of Technology, 2020.

[7] L. Mironova. X-tee platvormi kriitiliste kesksete teenuste testimine (Testing of the critical central services of the X-Road platform). Master's thesis, Tallinn University of Technology, 2017.

[8] P. Kivimäki. Resisting Failure. NIIS blog, 20 November 2020. `https://www.niis.org/blog/2020/11/20/resisting-failure`.

[9] Estonian Information System Authority (RIA). X-tee Factsheet, Estonia. `https://www.x-tee.ee/factsheets/EE/`. Figures based on data of 1 September 2026; accessed 17 September 2026.

[10] Nordic Institute for Interoperability Solutions. X-Road: Security Server Architecture. `https://docs.x-road.global/Architecture/arc-ss_x-road_security_server_architecture.html`. Accessed 17 September 2026.

[11] Nordic Institute for Interoperability Solutions. X-Road: System Parameters (UG-SYSPAR). `https://docs.x-road.global/Manuals/ug-syspar_x-road_v6_system_parameters.html`. Accessed 17 September 2026.

[12] D. Naylor, A. Finamore, I. Leontiadis, Y. Grunenberger, M. Mellia, M. Munafò, K. Papagiannaki, P. Steenkiste. The Cost of the "S" in HTTPS. In *CoNEXT '14*, pp. 133–140, ACM, 2014. DOI: 10.1145/2674005.2674991.

[13] A. Georges, D. Buytaert, L. Eeckhout. Statistically Rigorous Java Performance Evaluation. In *OOPSLA '07*, pp. 57–76, ACM, 2007. DOI: 10.1145/1297027.1297033.

[14] C. Paquin, D. Stebila, G. Tamvada. Benchmarking Post-quantum Cryptography in TLS. In *PQCrypto 2020*, LNCS vol. 12100, pp. 72–91, Springer, 2020. Cryptology ePrint Archive 2019/1447.

[15] S. Hemminger. Network Emulation with NetEm. In *linux.conf.au 2005*, Canberra, 2005.